CARTS: Change-Aware Regression Test Selection for LLM Applications

Seo Won Yi
University of Toronto
Canada

Abstract

Teams maintaining LLM applications routinely rerun full evaluation suites after every prompt or configuration change. This practice is expensive yet often unnecessary, because many changes affect only narrow slices of the suite. We study *change-aware regression test selection*: given a typed change to a prompt artifact, predict which tests are likely impacted and selectively rerun only those. We present CARTS, a pipeline that constructs version-level ground truth across evolving prompt versions, classifies prompt diffs into typed changes, predicts impacted tests via rule-based or learned selectors, and adds random *sentinels*—a small sample of non-selected tests used as a safety check—as a conservative fallback. Across three benchmark domains (210 prompt versions, 270 total tests, ~19k labeled outcomes), the rule-based selector achieves R_{+s} of 0.62–0.94 while reducing evaluation calls by 0.37–0.47. Learned tree-based selectors improve markedly on harder domains: on Domain B, XGBoost reaches $R_{+s} = 1.00$ at CR = 0.60, compared to 0.66 for the rule baseline on the same version-holdout slice. Cross-domain transfer is often poor, indicating that each application generally requires its own trained model.

Keywords

large language models, regression testing, test selection, prompt engineering, continuous integration

1 Introduction

LLM-backed applications evolve continuously. Teams revise system prompts, few-shot demonstrations, safety policies, tool descriptions, output formats, and model settings in response to product requirements and model drift. The common operational response to any such change is to rerun the full evaluation suite. This is safe but expensive.

The cost problem is acute when suites are nontrivial. A suite may contain hundreds of tests, each requiring an LLM call, and every pull request that touches a prompt triggers a full sweep. Yet many reruns are redundant. A format-only edit should primarily affect schema-sensitive tests, and a localized workflow rewrite need not require exhaustive reruns of every retrieval or formatting case. Current evaluation tooling—including Promptfoo [6] and LangSmith [7]—provides useful evaluation infrastructure but does not automatically infer which tests are impacted by a specific change. Meanwhile, classical regression test selection assumes code artifacts such as control-flow or dependency graphs, which do not exist in the same form for prompt-centric LLM systems [1, 2].

This gap motivates *change-aware regression test selection* for LLM applications: given a typed prompt/configuration change derived from the diff between two prompt versions, can we conservatively rerun only the tests likely to be affected, rather than the entire suite? The key challenge is asymmetry of risk: running extra tests is costly but acceptable; missing a true regression is much more serious.

We present CARTS, a change-aware regression test selector for LLM applications. The core idea is to view prompt edits as *typed artifact changes*—for example, modifications to `format`, `policy`, `workflow`, or `demonstration` sections—and to map those changes to candidate impacted tests. CARTS supports a transparent rule-based selector and a learned selector (LightGBM, XGBoost, logistic regression), both augmented with random *sentinel* sampling as a conservative safety mechanism. Sentinels are tests drawn at random from the non-selected pool; any sentinel that catches a regression triggers escalation to a full rerun.

This work makes four contributions:

- We formalize change-aware regression test selection for LLM applications as conservative impacted-test prediction conditioned on typed prompt diffs.
- We build an end-to-end pipeline with typed change classification, offline test tagging, rule-based and learned selectors, and sentinel-based fail-safes.
- We construct a three-domain benchmark with 210 prompt versions and evaluate the system under version-holdout, cross-domain, and grouped cross-validation protocols.
- We provide an empirical analysis of where selective rerun works, where it breaks, and how safety depends on sentinels and domain-specific training.

2 Problem Formulation

Let P denote the current prompt/configuration and P' a modified version. Let $T = \{t_1, \ldots, t_N\}$ be the evaluation suite. Let $\Delta(P, P')$ denote the set of typed changes extracted via section-aware comparison of P and P'. For each test t_i, let $\text{outcome}(P, t_i) \in \{\text{pass}, \text{fail}\}$.

We define the *impact set*

$$I(\Delta) = \left\{t_i \in T : \text{outcome}(P, t_i) \neq \text{outcome}(P', t_i)\right\}. \tag{1}$$

A selector returns $S(\Delta) \subseteq T$. The ideal objective is

$$\min |S(\Delta)| \quad \text{subject to} \quad I(\Delta) \subseteq S(\Delta), \tag{2}$$

that is, skip as many tests as possible without omitting any truly impacted test. Because $I(\Delta)$ is unknown before rerunning, the selector must predict it from diff metadata and test metadata alone.

We report five metrics. **Predictor recall** (R_{pred}) measures the fraction of impacted tests selected before sentinels are added. **Recall with sentinel** (R_{+s}) measures recall after unioning the predictor's set with the sentinel sample. **Call reduction** (CR) is $1 - |S(\Delta)|/|T|$. **False omission rate** (FOR) is the fraction of non-selected tests that were actually impacted. **Sentinel catch rate** measures how often sentinels catch at least one missed impacted test when the predictor has omitted some.

This problem is distinct from benchmark compression, which asks for a *fixed* subset approximating full-benchmark rankings across evaluations. Our setting is *change-conditional*: the selected subset varies from version to version depending on which artifacts changed.

3 Related Work

Classical regression test selection has a long history in software engineering. Safe and efficient selection criteria based on code structure were formalized by Rothermel and Harrold [1], and subsequent surveys synthesized selection, minimization, and prioritization techniques across a broad range of settings [2]. These methods rely on code-level artifacts such as control-flow graphs and dependency analysis; they do not directly transfer to prompt-centric LLM applications, where the "source code" is natural-language configuration.

Recent work argues that evolving LLM applications need regression testing in their own right. Ma et al. show that prompt and model evolution can silently degrade application behavior and argue that traditional testing assumptions must be rethought for LLM APIs [3]. RETAIN studies interactive regression testing for LLM migration scenarios [4], while ReCatcher focuses on regression testing for LLM code generation across model updates [5]. These efforts establish the need for LLM regression testing, but they do not address the specific problem studied here: *automatic change-aware selective rerun* conditioned on typed prompt and configuration edits.

At the tooling level, Promptfoo [6] supports evaluation pipelines and metadata-based filtering, and LangSmith [7] provides prompt versioning, diffing, and rollback workflows. These are natural integration targets for a selector such as CARTS, but they do not themselves infer impacted tests from a change diff.

4 System Design

4.1 Overview

CARTS takes as input an old prompt P, a modified prompt P', and an evaluation suite T. The pipeline has five stages:

(1) **Change classification.** Parse P and P' into sections and compute a structured diff to produce typed changes with attributes such as `unit_type`, `change_type`, and `magnitude`.
(2) **Test tagging.** Enrich each test with sensitivity metadata: monitor type, key dependencies, and prompt-section sensitivities.
(3) **Impact prediction.** Use either a rule-based affinity table or a learned model to predict which tests are likely impacted.
(4) **Sentinel sampling.** Draw $\max(5, \lceil 0.10 \times \text{pool_size} \rceil)$ tests at random from the non-predicted pool as probes for missed regressions.
(5) **Selection and escalation.** Rerun the union of predicted tests and sentinels. If any sentinel catches an unexpected regression, escalate to a full rerun.

4.2 Typed Change Taxonomy

Rather than treating a prompt edit as raw text, CARTS models the structured diff between prompts as a set of typed artifact changes. Unit types include `role`, `format`, `demonstration`, `demo_item`, `policy`, and `workflow`. Each change is further annotated with a `change_type` (inserted, deleted, modified, or reordered) and a magnitude estimated from normalized token-level Levenshtein distance. For format edits, the classifier also extracts renamed, inserted, or deleted output keys; for demonstrations, item-level diffing isolates changes to specific examples rather than the section as a whole.

This typed representation matters because different change families tend to affect different parts of the suite: format-key changes plausibly affect schema and required-key tests, whereas demonstration-item edits may affect only few-shot-sensitive slices. Typed changes therefore offer a better basis for selective rerun than undifferentiated text similarity.

4.3 Rule-Based Selector

The rule-based selector matches change types to test sensitivities via an affinity table, then augments the predicted set with sentinels. It is deliberately transparent: each predicted test carries a human-readable reason for inclusion. This makes the rule-based selector useful both as a competitive baseline and as a cold-start policy when no training data is available.

4.4 Learned Selector

The learned selector predicts an impact probability for each version, test pair using 42 features across three groups:

- **Change features (15):** one-hot unit/change types, edit magnitude, counts of affected keys and demonstration labels, compound-change indicators, and PCA-compressed change-diff embeddings.
- **Test features (22):** monitor-type and sensitivity-category indicators, dependency counts, prompt-section sensitivity flags, and PCA-compressed test-input embeddings.
- **Pairwise features (5):** change-test cosine similarity, key overlap, demonstration-label overlap, section-family match, and a rule-based prediction indicator.

Text embeddings are produced with `all-MiniLM-L6-v2` [10] and reduced to 24 PCA components per embedding group. We evaluate three model families: LightGBM [8], XGBoost [9], and logistic regression. Hyperparameters for tree models are tuned via Optuna [11] (50 trials per configuration); decision thresholds are selected from out-of-fold predictions on the training partition, targeting a minimum recall of 0.95. A critical design constraint is *no oracle leakage*: at prediction time, the model sees only information available in a real deployment (old/new prompts, test metadata, and learned weights), never ground-truth impact labels.

5 Experimental Setup

5.1 Benchmark

The benchmark consists of three hand-built domains (A, B, C), each representing a distinct class of LLM application with a structured prompt, an evaluation suite, and 70 prompt versions generated via controlled mutations.

Domain descriptions. **Domain A (JSON Extraction)** models a structured information extraction system with strict output constraints. The prompt includes role, format, demonstrations, policy, and workflow sections; the format section specifies a JSON schema

with required keys and serialization rules, and demonstrations provide few-shot examples across multiple customer-support scenarios. The evaluation suite contains 100 test cases covering schema validation, key-value correctness, demonstration coverage, policy compliance, and format adherence. Tests are explicitly sensitive to prompt components such as format and demonstrations, creating localized dependencies between changes and evaluation outcomes.

Domain B (Coding Assistant) represents a code-generation assistant with structured outputs and multi-step reasoning requirements. The prompt includes role definition, output format conventions (code and explanation), demonstrations across programming tasks, workflow decomposition, and policy constraints. The evaluation suite contains 80 test cases assessing functional correctness via code execution, format compliance, explanation quality, domain coverage, and safety policies. Compared to Domain A, this domain exhibits stronger coupling between workflow, demonstrations, and output correctness, making impact propagation more diffuse across tests.

Domain C (Hospital Intake Assistant) models a clinical triage assistant that processes patient symptom descriptions and outputs a structured JSON object for routing and prioritization. The system operates under a non-diagnostic constraint, focusing on categorization and triage rather than medical advice. The prompt includes role, format, demonstrations, policy, and workflow sections; the format enforces a strict schema with fixed keys (e.g., `urgency`, `department`, `follow_up_questions`) and enumerated values, and the policy encodes safety constraints such as crisis handling and PII restrictions. The evaluation suite contains 90 test cases covering schema correctness, clinical routing, policy compliance, and adversarial inputs. This domain exhibits a bimodal impact distribution—many versions produce no observable changes, while a subset cause large regressions—making it a challenging but distinct setting for selective rerun.

Prompt version generation. Each domain contains 70 prompt versions generated through controlled, typed mutations applied to a base prompt, organized into seven mutation categories with fixed version ranges. Versions v01–v15 cover format and schema mutations: systematic key renames, field insertions and removals, and serialization tweaks such as wording and layout changes to schema descriptions. Versions v16–v29 cover demonstration mutations: removal of specific few-shot examples by index, substitution of new scripted examples at designated positions, and surgical string edits within individual example outputs (e.g., flipping a priority value or changing a triage field in one demo). Versions v30–v45 cover policy, workflow, and role mutations: phrase strengthening and weakening with clause additions and removals (v30–v36); step reordering, addition, and removal (v37–v41); and tone and phrasing shifts (v42–v45). Versions v46–v50 are compound mutations that stack two or three atomic mutators in sequence, producing change logs with multiple typed changes (e.g., key rename combined with policy strengthening, or demo removal combined with workflow reordering).

Versions v51–v58 are LLM-generated using a fixed set of `(section, instruction)` pairs to prompt `gpt-4o-mini` to rewrite one section at a time (v51–v55), with an additional semantic rewrite variant that stresses meaning change while preserving structural elements such as bullets and headings (v56–v58). These versions are not open-ended explorations; they are curated scenario edits designed to mimic realistic, messier human prompt iteration.

Table 1: Benchmark scale and full-rerun labeling cost. Calls include one baseline reference run per domain in addition to the 70 version runs ($(\text{versions} + 1) \times \text{tests}$). Base pass is the fraction of tests passing on the unmodified base prompt.

Domain	Tests	Versions	Calls	Cost	Base pass
A	100	70	7,100	$0.95	0.95
B	80	70	5,680	$1.15	0.82
C	90	70	6,390	$1.52	0.94
Total	270	210	19,170	$3.62	–

Three final categories simulate increasingly challenging conditions. Versions v59–v61 introduce *intentional cross-section inconsistencies*, where one section is updated but a related section is left stale—for example, a key renamed in the format section but not in demonstrations, or a workflow step reordered to contradict demo ordering. These versions directly simulate prompt drift and cross-section incoherence that arise in practice. Versions v62–v65 cover extended mutation types: appended meta-instructions (e.g., "think step by step before outputting JSON"), injected chain-of-thought blocks, changed dialogue labels (e.g., `User/Assistant` → `Input/Output`), and uppercased emphasis phrases mimicking "PLEASE FOLLOW" edits. Versions v66–v70 are *sequential chains*, where a second atomic mutation is applied on top of an already-mutated version (e.g., starting from v01's key-renamed sections and adding a policy strengthening on top), modeling realistic iterative editing in which one fix prompts a follow-on change.

Each mutation is applied at the section or item level and produces a structured diff with explicit `unit_type`, `change_type`, and `magnitude` attributes, where magnitude is the normalized token-level Levenshtein distance between old and new section text. Together, this structure ensures coverage across change types, magnitudes, and edit patterns ranging from localized mechanical changes to cross-sectional semantic rewrites and dependent multi-hop edits.

Ground truth and scale. Table 1 summarizes the scale and labeling cost of each domain. Ground truth is produced by full rerun under a fixed evaluator model (`gpt-4o-mini` via the OpenAI Batch API). For each version, the full suite is executed and outcomes are compared against a baseline reference, defined as the immediately preceding prompt version or a cached reference run for v01. A test is considered impacted if its pass/fail outcome differs between the baseline and the modified version. Because each domain requires one baseline run in addition to the 70 version runs, the total call count per domain is $(\text{versions} + 1) \times \text{tests}$ (e.g., $71 \times 100 = 7{,}100$ for Domain A). This procedure yields 19,170 labeled $(\text{version}, \text{test})$ outcomes across 210 prompt versions.

The three domains exhibit distinct impact regimes. Domain A shows frequent but moderate-sized changes: 67 of 70 versions have at least one impacted test, with a mean of 10.86 impacted tests per version. Domain B is the densest, with 69 of 70 impacted versions, though the mean impact count is smaller at 6.20, reflecting its more consistently distributed but diffuse impact structure. Domain C is

bimodal: 46 of 70 versions have zero impacted tests, yet impacted versions can be severe (mean 7.13, maximum 35). This bimodality helps explain why Domain C is relatively easy for the rule baseline in aggregate while still containing hard cases.

5.2 Baselines and Evaluation Protocols

We compare against three non-learned baselines. The *full rerun* baseline selects all tests (CR = 0, R_{+s} = 1.0) and serves as the oracle coverage reference; per-version recall is defined as 1.0 even when no tests are impacted, avoiding a 0/0 case. *Random 50% selection* samples a fixed half of the suite (floor division, seed 42) reused identically across all versions in a domain, providing a stable but change-agnostic lower bound on call reduction. The *monitor-family heuristic* maps each detected change unit type to an allowed set of monitor types via a fixed affinity table (e.g., `format` → schema/required-key monitors; `policy` → policy monitors); if `role` appears among the changed units, the heuristic selects all tests conservatively. The rule-based selector is evaluated over all 70 versions per domain with sentinel fraction 0.10, minimum sentinel count 5, and default magnitude threshold 0.005; sentinels are drawn exclusively from tests not already selected by the predictor.

For the learned selector, we report three protocols:

- **Version holdout:** train on versions v01–v50 and evaluate on v51–v70 (20 held-out versions per domain).
- **Cross-domain transfer:** train on one domain and evaluate on another without retraining.
- **5-fold GroupKFold:** grouped by version to prevent leakage, with both single-domain and combined multi-domain training. In the combined setting, training rows include all versions from the held-in folds of the target domain plus all rows from the other two domains; the test fold contains only held-out versions from the target domain.

For the version holdout and single-domain k-fold, statistical comparisons between learned and rule-based selectors use one-sided paired Wilcoxon signed-rank tests on per-version R_{+s} vectors aligned by version ID, alongside 95% bootstrap confidence intervals (10,000 resamples, seed 42) on the mean paired difference. Cross-domain transfer results are reported descriptively, as differing domain structures preclude a meaningful paired comparison against a single-domain rule baseline.

6 Results

6.1 Rule-Based Baseline

Table 2 reports aggregate rule-based performance across all 70 versions per domain. The rule selector is already competitive on Domain C, achieving R_{+s} = 0.94 with CR = 0.47 and FOR = 0.007. Domains A and B are substantially harder: R_{+s} falls to 0.65 and 0.62, respectively, with higher false omission rates.

These averages mask strong heterogeneity by change type. Role changes are handled conservatively by selecting all tests, yielding perfect recall but zero savings. In Domain A, policy changes are a failure case (R_{+s} = 0.13 at CR = 0.76), while workflow changes are largely rescued by sentinels (R_{pred} = 0.29 rising to R_{+s} = 0.93). In Domain B, workflow changes remain poor even after sentinels (R_{+s} = 0.09). Domain C is consistently easier across most change-type strata. Full per-domain change-type tables are provided in Appendix A. This heterogeneity motivates richer learned selectors that can capture cross-section effects beyond direct rule matches.

Table 2: Aggregate rule-based performance (all 70 versions per domain).

Domain	R_{pred}	R_{+s}	CR	FOR	Perfect
A	0.57	0.65	0.46	0.013	35/70
B	0.54	0.62	0.37	0.030	30/70
C	0.93	0.94	0.47	0.007	64/70

6.2 Version Holdout

Table 3 compares learned selectors against the rule baseline on the version holdout (v51–v70). All three learned models outperform rules on Domains A and B. The most striking result is Domain B, where XGBoost achieves R_{+s} = 1.00 with CR = 0.60 and FOR = 0.000, versus rule-based R_{+s} = 0.662 on the same versions. On Domain C, the rule baseline is already near saturation (R_{+s} = 0.925); learned trees roughly match it, while logistic regression lags (R_{+s} = 0.80).

Training diagnostics and bootstrap confidence intervals are provided in Appendix B (Tables 10 and 11). All tree models achieve perfect training AUROC; the generalization gap is smallest on Domain C (test AUROC 0.997) and largest on Domain A (0.826–0.880). Logistic regression's substantially lower test AUROC on Domain B (0.685 vs. 0.936–0.946 for trees) explains its low call reduction in Table 3: it preserves recall only by over-selecting (CR = 0.28). Statistically, version-holdout gains on Domain A do not reach significance at $\alpha = 0.05$ (d = 0.14–0.21), whereas Domain B improvements are both significant and large ($p \leq 0.002$, d = 0.84–0.97). Domain C comparisons are mostly non-significant because the rule baseline is already near saturation. The complete set of paired comparisons is given in Appendix D.

6.3 Cross-Domain Transfer

Cross-domain transfer is the clearest negative result in the study. Table 4 shows the compact R_{+s} matrix for XGBoost per direction; Table 5 gives the complete per-model breakdown. Transfer *to* Domain C is relatively strong (A→C: R_{+s} = 0.87; B→C: 0.83), but transfer *from* Domain C collapses in both directions ($R_{+s} \leq 0.28$), and transfer between Domains A and B is also weak (A→B: 0.35; B→A: 0.44). The cross-domain AUROC data (Table 12 in Appendix C) confirms the pattern: C→B drops to 0.495 (XGBoost) and 0.448 (logistic regression)—at or below random-guess level—matching the near-zero predictor recall in Table 5.

Several patterns emerge from Table 5. For transfers *to* Domain A, B→A with logistic regression achieves partial success (R_{+s} = 0.71), but its FOR of 0.171 reveals that a substantial fraction of non-selected tests were actually impacted. C→A stays weak on recall (R_{+s} = 0.20–0.28) despite high call reduction, with sentinels providing most of the safety coverage. For transfers *to* Domain B, C→B is a comprehensive failure: predictor recall falls to 0.01–0.02 across all model families, and observed R_{+s} of 0.27–0.28 is almost entirely sentinel-driven. A→B with logistic regression is a deceptive case:

Table 3: Version-holdout results (train v01–v50, test v51–v70).

Domain	Model	R_{pred}	R_{+s}	CR	FOR	Sentinel catch
A	LightGBM	0.83	0.84	0.82	0.019	0.20
A	XGBoost	0.82	0.82	0.82	0.019	0.20
A	LogReg	0.86	0.86	0.76	0.021	0.20
B	LightGBM	0.95	0.95	0.70	0.005	0.00
B	XGBoost	0.98	**1.00**	0.60	**0.000**	1.00
B	LogReg	0.90	0.91	0.28	0.007	0.33
C	LightGBM	0.92	0.93	0.83	0.001	0.33
C	XGBoost	0.98	0.98	0.80	0.001	0.00
C	LogReg	0.80	0.80	0.82	0.003	0.20

Table 4: Compact cross-domain transfer matrix: R_{+s} for XGBoost.

Train ↓ / Test →	A	B	C
A	–	0.35	0.87
B	0.44	–	0.83
C	0.28	0.28	–

high R_{+s} (= 0.93) but negligible call reduction (CR = 0.06) means the model is effectively running a near-full suite. Transfers *to* Domain C are the most consistent: A→C and B→C tree models achieve R_{+s} of 0.83–0.93 with meaningful call reduction (0.64–0.81), reflecting Domain C's structurally regular impact patterns. However, logistic regression for B→C shows elevated FOR (0.172) despite a respectable R_{+s} of 0.87, indicating that risk is concentrated in the non-selected pool. Taken together, these results indicate that feature distributions differ materially across applications and that each domain generally requires its own selector.

6.4 Grouped Cross-Validation and Feature Analysis

When more labeled history is available, grouped cross-validation presents a more optimistic picture. Table 6 reports the complete 5-fold GroupKFold results for all three model families under both single-domain and combined multi-domain training.

Single-domain tree models achieve R_{+s} of 0.92–0.97 across all three domains with CR of 0.68–0.81, substantially outperforming the version holdout on the same domains. Logistic regression shows a consistent pattern: high R_{+s} (0.88–0.96) but substantially lower call reduction (0.31–0.69), reflecting the same over-selection tendency observed in version-holdout evaluation.

Combined multi-domain training raises recall stability at the cost of call reduction. The trade-off is clearest on Domain A: combined LightGBM achieves R_{+s} = 0.97 at CR = 0.47, versus single-domain LightGBM at R_{+s} = 0.92 with CR = 0.70. The extra data volume from the other two domains improves recall but introduces feature noise that reduces the model's ability to prune aggressively. The exception is Domain C, where combined XGBoost achieves the best joint result across all configurations (R_{+s} = 1.00, CR = 0.78, FOR = 0.002), suggesting that the additional data from Domains A and B is informative rather than noisy for this domain's structured impact patterns. These results show that the learned selector is data-efficient once a domain has accumulated sufficient labeled history, supporting a practical adoption path: start with the transparent rule-based selector, accumulate labels conservatively, then train a domain-specific learned model.

Feature importance analysis (LightGBM split importance) reveals structurally different predictive signals across domains. On Domains A and C, *key overlap*—measuring how many output keys referenced by a test are also touched by the change—dominates by a large margin, indicating that localized key dependencies between format edits and schema-sensitive tests are the primary predictive signal. Domain B differs markedly: the *number of affected demonstration labels* is the top feature, reflecting a suite more dependent on few-shot examples than on output-key alignment. Across all domains, PCA components from text embeddings and section-match indicators also contribute meaningfully, confirming that both symbolic dependency features and learned semantic features are useful.

7 Discussion

When selection works. Change-aware selection is most effective when tests can be tied to concrete artifact dependencies. Domain C exemplifies this: format edits align tightly with schema-sensitive tests, *key overlap* is highly informative, and even simple rules achieve R_{+s} = 0.94. With learned models and sufficient training history, call reductions of 0.47–0.83 are achievable while maintaining $R_{+s} \geq 0.93$.

When it fails. Two failure modes dominate. The first is *diffuse impact*: policy, workflow, and some demonstration edits can influence behavior in cross-sectional ways that static rules miss entirely (Domain A policy: R_{+s} = 0.13; Domain B workflow: R_{+s} = 0.09 under rules). Learned models can mitigate this when domain-specific training data is available, as shown by XGBoost's perfect recall on Domain B. The second is *distribution shift*: models trained on one domain frequently fail to generalize to another (C→B: R_{+s} = 0.28), and high R_{+s} in such settings can mask weak localization when safety is supplied primarily by sentinel coverage rather than the predictor itself.

Operational implications. Three deployment implications follow from these results. First, *rule-based cold start is practical*: the rule selector requires no training data and provides moderate savings with conservative behavior, making it suitable for immediate deployment.

Table 5: Full cross-domain transfer results (all model families, 70 test versions per target domain). Results are grouped by evaluation target domain. A→B with logistic regression achieves high R_{+s} (0.93) but selects 94% of the suite (CR = 0.06), making it operationally useless as a selector.

Train	Eval	Model	R_{pred}	R_{+s}	CR	FOR
B	A	LightGBM	0.23	0.36	0.75	0.039
B	A	XGBoost	0.21	0.44	0.76	0.059
B	A	LogReg	0.68	0.71	0.52	0.171
C	A	LightGBM	0.23	0.27	0.82	0.028
C	A	XGBoost	0.24	0.28	0.81	0.027
C	A	LogReg	0.13	0.20	0.86	0.076
A	B	LightGBM	0.08	0.32	0.83	0.073
A	B	XGBoost	0.13	0.35	0.80	0.063
A	B	LogReg	0.91	0.93	0.06	0.012
C	B	LightGBM	0.01	0.27	0.90	0.072
C	B	XGBoost	0.02	0.28	0.88	0.073
C	B	LogReg	0.01	0.27	0.90	0.072
A	C	LightGBM	0.93	0.93	0.77	0.006
A	C	XGBoost	0.87	0.87	0.81	0.007
A	C	LogReg	0.96	0.96	0.16	0.002
B	C	LightGBM	0.86	0.86	0.64	0.014
B	C	XGBoost	0.82	0.83	0.77	0.019
B	C	LogReg	0.81	0.87	0.43	0.172

Table 6: 5-fold GroupKFold results (all model families). *Single-domain*: trained and evaluated on the target domain only. *Combined*: trained on all three domains' versions, evaluated on the target domain only. Each version appears in exactly one test fold; groups are defined at the version level to prevent leakage. Combined XGBoost on Domain C is the peak joint recall/efficiency result across all configurations.

Domain	Training	Model	R_{pred}	R_{+s}	CR	FOR
A	Single	LightGBM	0.92	0.92	0.70	0.012
A	Single	XGBoost	0.92	0.92	0.71	0.009
A	Single	LogReg	0.96	0.96	0.31	0.007
B	Single	LightGBM	0.90	0.92	0.70	0.010
B	Single	XGBoost	0.91	0.93	0.68	0.010
B	Single	LogReg	0.84	0.88	0.40	0.014
C	Single	LightGBM	0.96	0.97	0.81	0.008
C	Single	XGBoost	0.96	0.97	0.80	0.007
C	Single	LogReg	0.90	0.90	0.69	0.002
A	Combined	LightGBM	0.97	0.97	0.47	0.005
A	Combined	XGBoost	0.93	0.93	0.72	0.007
A	Combined	LogReg	0.98	0.98	0.39	0.005
B	Combined	LightGBM	0.97	0.98	0.42	0.013
B	Combined	XGBoost	0.96	0.97	0.58	0.009
B	Combined	LogReg	0.91	0.92	0.26	0.008
C	Combined	LightGBM	0.98	0.98	0.70	0.005
C	Combined	XGBoost	**1.00**	**1.00**	**0.78**	0.002
C	Combined	LogReg	0.90	0.92	0.63	0.004

Second, *each application needs its own model*: cross-domain transfer is too unreliable for plug-and-play deployment across unrelated prompt applications. Third, *sentinel-based escalation is essential*: a production system should treat any sentinel-detected failure as evidence of model staleness and escalate to a full rerun.

Toward online refinement. The learned selector evaluated here is trained offline on a fixed ground-truth dataset. In production, however, every CI run that executes a test produces a fresh change, test, impacted observation, creating an opportunity for a *predict–measure–retrain* loop: a developer submits a prompt change; the selector predicts impacted tests and runs them along with sentinels; the observed outcomes are logged as new training examples; and the model is periodically retrained on the accumulated data. A key challenge is *partial observability*: because skipped tests produce

no signal, the model risks reinforcing its own blind spots over time. Three mechanisms mitigate this. Sentinel sampling acts as an exploration arm, providing ground truth from the non-selected pool and surfacing model blind spots. Periodic full reruns—for example, every 10th–20th pull request—supply uncensored calibration data at modest cost while maintaining model freshness. Uncertainty-driven selection (active learning) can further prioritize tests where the model is least confident, maximizing the information gained from each run. This online extension is complementary to the offline evaluation presented here and represents a natural path toward a self-improving selector.

Cost considerations. Cost projections should be treated as illustrative rather than definitive, since the appropriate parameters depend on suite size, call frequency, and observed selector performance in a specific deployment. Under snapshot assumptions of call reduction $r = 0.9$ and escalation rate $e = 0.3$, the savings fraction is $r(1 - e) = 0.63$; as a concrete example, a 500-test suite at \$0.05 per call with 200 PRs per month saves roughly \$3,150 monthly. The version-holdout learned models in our experiments actually achieve CR of 0.28–0.83, implying an illustrative savings range of roughly 0.20–0.58 under the same escalation assumption—notably wider than the snapshot figure. The qualitative conclusion remains useful: savings scale linearly with suite size and call reduction, but deployment-specific projections should be recomputed from current selector behavior rather than copied from any fixed-parameter table.

8 Threats to Validity

- **Synthetic benchmark.** Although designed to reflect realistic prompt changes, our domains and mutations are controlled. Real production change logs may exhibit materially different distributions.
- **Small holdout slices.** Version-holdout evaluation uses only 20 versions per domain, making paired significance tests underpowered, particularly when the rule baseline is already strong.
- **Sentinel-dependent safety.** High R_{+s} can coexist with weak R_{pred} when sentinel coverage carries most of the recall load (e.g., C→B cross-domain: R_{pred} = 0.01–0.02 yet R_{+s} = 0.28). Aggregate metrics do not always distinguish genuine impact localization from randomized safety coverage, so R_{pred} and FOR should be consulted alongside R_{+s} when assessing selector quality.
- **Illustrative cost analysis.** Cost projections use assumed parameter values and are useful for relative comparison, not as a substitute for deployment-specific measurement.
- **No online learning.** The predict-measure-retrain loop described in Section 7 is a design implication; this work evaluates only offline-trained models and does not implement or measure an adaptive system.

9 Conclusion

We presented CARTS, a change-aware regression test selection pipeline for LLM applications built around typed prompt diffs, rule-based and learned impact prediction, and sentinel-based conservative fallbacks. Across three benchmark domains, rule-based selection reduces calls by 0.37–0.47 while achieving R_{+s} of 0.62–0.94. Version-holdout learned tree models push call reduction to 0.60–0.83 while maintaining R_{+s} of 0.82–1.00. Learned tree models substantially outperform rules on harder domains when domain-specific training data is available, but cross-domain transfer is unreliable, and strong results sometimes depend on sentinel coverage rather than genuine impact localization.

Our findings support a pragmatic adoption strategy: deploy transparent rules first, collect labels conservatively through selective reruns augmented by periodic full reruns, then train domain-specific learned selectors once sufficient history has accumulated. Future work should validate this approach on real prompt-revision logs, implement the online predict-measure-retrain loop outlined in Section 7, and study how model-version changes interact with prompt-diff-driven regression testing.

References

[1] Gregg Rothermel and Mary Jean Harrold. 1997. A Safe, Efficient Regression Test Selection Technique. *ACM Transactions on Software Engineering and Methodology* 6, 2 (1997), 173–210.

[2] Shin Yoo and Mark Harman. 2012. Regression Testing Minimization, Selection and Prioritization: A Survey. *Software Testing, Verification and Reliability* 22, 2 (2012), 67–120.

[3] Wanqin Ma, Chenyang Yang, and Christian Kästner. 2024. (Why) Is My Prompt Getting Worse? Rethinking Regression Testing for Evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN)*. 166–171.

[4] Tanay Dixit, Daniel Lee, Sally Fang, Sai Sree Harsha, Anirudh Sureshan, Akash Maharaj, and Yunyao Li. 2024. RETAIN: Interactive Tool for Regression Testing Guided LLM Migration. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*.

[5] Altaf Allah Abbassi, Léuson M. P. da Silva, Amin Nikanjam, and Foutse Khomh. 2025. ReCatcher: Towards LLMs Regression Testing for Code Generation. *CoRR* abs/2507.19390.

[6] Promptfoo. 2026. Metadata Filtering in Promptfoo Evaluation Pipelines. Documentation. https://www.promptfoo.dev/docs/configuration/guide/

[7] LangSmith. 2026. Manage Prompts in LangSmith. Documentation. https://docs.langchain.com/langsmith/

[8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems* 30.

[9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794.

[10] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3982–3992.

[11] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2623–2631.

A Rule-Based Performance by Change Type

Tables 7–9 break down rule-based selector performance by primary change type. Column n is the number of versions whose primary change matches the row. Sentinel catch rate (Sent.) measures how often sentinels recover at least one missed impacted test; a dash (—) indicates either perfect predictor recall or no impacted tests in that stratum.

Policy changes are the hardest for rules on Domain A (R_{+s} = 0.13, sentinel catch rate 0.00) and are largely rescued by sentinels on Domain B (R_{pred} = 0.38 → R_{+s} = 0.83, catch rate 0.91). Workflow changes remain unsalvageable on Domain B (R_{+s} = 0.09).

Table 7: Rule-based performance by change type, Domain A.

Type	n	R_{pred}	R_{+s}	CR	FOR	Sent.
compound	18	0.97	0.98	0.15	0.001	0.90
demo_item	5	0.40	0.47	0.84	0.016	0.33
demonstration	10	0.49	0.49	0.72	0.017	0.00
format	11	0.40	0.45	0.51	0.031	0.10
policy	11	0.13	0.13	0.76	0.024	0.00
role	8	1.00	1.00	0.00	0.000	—
workflow	7	0.29	0.93	0.54	0.005	0.83

Table 8: Rule-based performance by change type, Domain B.

Type	n	R_{pred}	R_{+s}	CR	FOR	Sent.
compound	19	0.86	0.90	0.11	0.025	0.70
demo_item	5	0.40	0.40	0.61	0.025	0.00
demonstration	10	0.34	0.34	0.50	0.043	0.00
format	9	0.35	0.35	0.25	0.072	0.00
policy	11	0.38	0.83	0.79	0.007	0.91
role	8	1.00	1.00	0.00	0.000	—
workflow	8	0.09	0.09	0.61	0.043	0.00

Table 9: Rule-based performance by change type, Domain C.

Type	n	R_{pred}	R_{+s}	CR	FOR	Sent.
compound	22	0.91	0.92	0.36	0.020	0.50
demo_item	5	1.00	1.00	0.72	0.000	—
demonstration	6	0.67	0.67	0.72	0.005	0.00
format	10	0.99	1.00	0.55	0.003	0.50
policy	12	0.96	0.96	0.68	0.001	0.00
role	8	1.00	1.00	0.00	0.000	—
workflow	7	1.00	1.00	0.50	0.000	—

Domain C is uniformly strong, with the sole weakness on demonstration changes ($R_{+s} = 0.67$). Role changes trigger full-suite selection across all domains ($R_{pred} = 1.0$, CR = 0.0), reflecting the rule selector's conservative handling of this high-risk change type.

B Version-Holdout Training Diagnostics

Table 10 reports training and test AUROC alongside the tuned decision threshold for each model and domain on the version-holdout split. Table 11 gives bootstrap 95% confidence intervals for R_{+s} and CR on the 20-version holdout slice.

Tree models use very low thresholds (0.01–0.02) due to sharp probability estimates, whereas logistic regression requires higher thresholds on Domains A (0.08) and C (0.33). Domain B confidence intervals are tight (XGBoost: [1.000, 1.000] for R_{+s}), while Domain A shows wider intervals ([0.675, 0.955]), consistent with the larger generalization gap.

C Cross-Domain Training Diagnostics

Table 12 reports training and test AUROC for cross-domain model configurations referenced in Section 6. All tree models achieve perfect training AUROC on the source domain; the test AUROC on the target domain quantifies the transfer gap.

Table 10: Training diagnostics for the version-holdout split (train v01–v50). Decision thresholds are tuned per model and domain to optimise recall.

Domain	Model	Train AUROC	Test AUROC	Threshold
A	LightGBM	1.000	0.880	0.02
A	XGBoost	1.000	0.826	0.01
A	LogReg	0.994	0.842	0.08
B	LightGBM	1.000	0.936	0.01
B	XGBoost	1.000	0.946	0.01
B	LogReg	0.942	0.685	0.01
C	LightGBM	1.000	0.997	0.01
C	XGBoost	1.000	0.997	0.01
C	LogReg	0.996	0.928	0.33

Table 11: Bootstrap 95% confidence intervals for R_{+s} and CR on the version holdout (v51–v70), based on 10,000 resamples of the 20-version score vector.

Domain	Model	R_{+s} 95% CI	CR 95% CI
A	LightGBM	[0.702, 0.955]	[0.757, 0.860]
A	XGBoost	[0.675, 0.955]	[0.763, 0.870]
A	LogReg	[0.723, 0.973]	[0.687, 0.822]
B	LightGBM	[0.891, 0.991]	[0.626, 0.768]
B	XGBoost	[1.000, 1.000]	[0.506, 0.684]
B	LogReg	[0.775, 1.000]	[0.183, 0.400]
C	LightGBM	[0.800, 1.000]	[0.760, 0.891]
C	XGBoost	[0.925, 1.000]	[0.724, 0.858]
C	LogReg	[0.600, 0.950]	[0.712, 0.900]

Table 12: Cross-domain training and test AUROC. Train AUROC reflects source-domain fit; test AUROC reflects target-domain discriminative performance.

Source → Target	Model	Train AUROC	Test AUROC
A → B	LightGBM	1.000	0.555
A → B	XGBoost	1.000	0.680
A → C	LightGBM	1.000	0.975
A → C	XGBoost	1.000	0.978
B → A	LightGBM	1.000	0.842
B → A	XGBoost	1.000	0.859
C → A	LightGBM	1.000	0.910
C → B	LightGBM	1.000	0.672
C → B	XGBoost	1.000	0.495
C → B	LogReg	0.997	0.448

The transfer gap is asymmetric. Transfers *to* Domain C maintain high test AUROC (0.975–0.978 for A→C), consistent with Domain C's structured, regular impact patterns. Transfers *from* Domain C to Domain B collapse to at or below random-guess level (0.448–0.672), confirming the operational cross-domain recall results in Section 6.

D Full Statistical Comparisons

D.1 Learned vs. Rule-Based Selectors

Table 13 reports all paired Wilcoxon signed-rank tests comparing learned selectors to the rule-based baseline. Tests are one-sided (H_1: learned > rule). Effect size is Cohen's d computed on the paired difference vector; 95% bootstrap CIs use 10,000 resamples (seed 42) of the mean paired difference.

Significance tracks domain difficulty and sample size. Domain A version-holdout improvements are visible in means but not significant with only 20 paired observations; k-fold comparisons on 70 versions reach high significance (medium effects, $d = 0.54$–0.65). Domain B shows uniformly significant improvements across all models and protocols, with large version-holdout effect sizes ($d = 0.84$–0.97 for tree models). Domain C is mostly inconclusive: the rule baseline is near saturation, and logistic regression is the only learned model that performs *worse* than rules on the version-holdout slice (negative mean difference, $p = 0.949$).

D.2 Model vs. Model Comparisons

Table 14 compares learned model families head-to-head using the same paired Wilcoxon procedure (two-sided).

No tree-vs.-tree comparison reaches significance, confirming that LightGBM and XGBoost are interchangeable for this task. The tree-vs.-logistic-regression gap is most visible on Domain C, where the nonlinear models can exploit the bimodal impact structure that a linear decision boundary cannot capture. On Domains A and B, logistic regression compensates for lower discriminative power by over-selecting (lower call reduction), which preserves recall but reduces savings.

Table 13: Paired Wilcoxon tests: learned vs. rule-based R_{+s}. n is the number of versions paired by version ID. Mean diff = learned − rule. Boldface p-values are significant at $\alpha = 0.05$.

Domain	Model (split)	n	p	d	Mean diff	95% CI
A	LGBM (holdout)	20	0.165	0.17	+0.093	[−0.131, 0.334]
A	XGB (holdout)	20	0.211	0.14	+0.078	[−0.148, 0.325]
A	LogReg (holdout)	20	0.128	0.21	+0.113	[−0.105, 0.348]
A	LGBM (kfold)	70	**2.4e−5**	0.56	+0.270	[0.156, 0.383]
A	XGB (kfold)	70	**2.7e−5**	0.54	+0.268	[0.152, 0.383]
A	LogReg (kfold)	70	**3.1e−6**	0.65	+0.307	[0.197, 0.418]
B	LGBM (holdout)	20	**0.002**	0.84	+0.284	[0.146, 0.425]
B	XGB (holdout)	20	**0.001**	0.97	+0.338	[0.200, 0.488]
B	LogReg (holdout)	20	**0.034**	0.48	+0.246	[0.021, 0.450]
B	LGBM (kfold)	70	**1.4e−6**	0.68	+0.299	[0.197, 0.399]
B	XGB (kfold)	70	**8.6e−7**	0.73	+0.306	[0.211, 0.402]
B	LogReg (kfold)	70	**1.0e−4**	0.55	+0.252	[0.143, 0.359]
C	LGBM (holdout)	20	0.500	0.00	0.000	[−0.150, 0.150]
C	XGB (holdout)	20	0.159	0.22	+0.050	[0.000, 0.150]
C	LogReg (holdout)	20	0.949	−0.39	−0.125	[−0.275, 0.000]
C	LGBM (kfold)	70	0.261	0.10	+0.028	[−0.037, 0.094]
C	XGB (kfold)	70	0.140	0.11	+0.029	[−0.033, 0.094]
C	LogReg (kfold)	70	0.935	−0.17	−0.039	[−0.096, 0.011]

Table 14: Model-vs.-model Wilcoxon tests. Mean diff is first model minus second.

Comparison	n	p	Mean diff	Note
LGBM vs. XGB, A holdout	20	0.09	+0.015	n.s.
LGBM vs. XGB, A kfold	70	0.500	+0.001	Tied
LGBM vs. LogReg, A holdout	20	0.500	−0.021	Tied
LGBM vs. LogReg, A kfold	70	0.988	−0.037	LogReg higher
LGBM vs. XGB, B holdout	20	0.971	−0.054	XGB higher
LGBM vs. XGB, B kfold	70	0.600	−0.007	Tied
LGBM vs. LogReg, B holdout	20	0.231	+0.038	n.s.
LGBM vs. LogReg, B kfold	70	0.110	+0.047	n.s.
LGBM vs. LogReg, C holdout	20	0.051	+0.125	Borderline
LGBM vs. LogReg, C kfold	70	0.050	+0.066	Borderline